

PyGNSS-RT

Technical Design Document

Python-Based Real-Time PPP-AR GNSS Positioning System

Comprehensive Technical Specification with Full Source Code

Repository: https://github.com/georgisalene/pygnss_rt

Version: v1.4.0-GE

Python Files: 75 modules • **Lines of Code:** 46,945

Addisu Hunegnaw

GNSS Research Group

Institute for Earth Sciences and Space Geodesy

`pygnss_rt` — Python GNSS Real-Time Processing

January 2026

Abstract

This document provides a comprehensive technical specification for the `pygnss_rt` Python package, a production-grade framework for Real-Time Precise Point Positioning with Ambiguity Resolution (PPP-AR). The system integrates with the Bernese GNSS Software v5.4 for core geodetic computations while providing a sophisticated Python orchestration layer for data management, product acquisition, and operational automation.

Key Features:

- Multi-GNSS support: GPS (G) + Galileo (E) combination for optimal AR
- VMF3 Vienna Mapping Functions for tropospheric modeling
- CODE OSB (Observable-Specific Signal Biases) for phase bias consistency
- LAMBDA-based integer ambiguity resolution
- Automated product downloads from CDDIS, CODE, IGN
- ZTD to IWV conversion for meteorological applications
- Five international station networks (IGS, EUREF, GB, RGP, Supersites)

GitHub Repository: https://github.com/georgisalene/pygnss_rt

This document includes the complete source code with colorful syntax highlighting, mathematical formulations, and architectural diagrams for all major system components.

Contents

1	Introduction	5
1.1	Document Purpose and Scope	5
1.1.1	What This Document Provides	5
1.2	System Overview	5
1.3	Codebase Statistics	6
2	System Architecture	7
2.1	Package Structure	7
2.2	Data Flow Pipeline	8
3	Command-Line Interface	9
3.1	Overview	9
3.2	Daily PPP Command	10
3.3	Available Commands	11
4	Daily PPP Processor	12
4.1	Module Overview	12
4.2	Main Processor Class	13
4.3	Product Download Integration	15
5	Product Downloader	17
5.1	Module Overview	17
5.2	ProductDownloader Class	18
5.3	BIA/OSB Path Building	20
6	Bernese Processing Engine Runner	22
6.1	Module Overview	22
6.2	BPERunner Class	23
6.3	INP File Customization	25
7	Network Profiles	27
7.1	Module Overview	27
7.2	Network Profile Factory	28
8	Date and Time Utilities	31
8.1	GNSSDate Class	31
8.2	Conversion Functions	33
9	ZTD to IWV Conversion	35
9.1	Physical Background	35
9.1.1	Mathematical Formulation	35
9.2	Implementation	35

10 Station Management	40
10.1 Station Class	40
11 Mathematical Foundations	43
11.1 PPP-AR Observation Model	43
11.2 Kalman Filter State Vector	43
11.3 LAMBDA Ambiguity Resolution	43
11.4 VMF3 Mapping Functions	44
12 Bernese Configuration	45
12.1 YAML Configuration Format	45
12.2 GE Configuration Rationale	46
A Complete Module Listing	47
B Installation and Usage	49
B.1 Installation from GitHub	49
B.2 Basic Usage Examples	49
C References	50

List of Figures

2.1 PyGNSS-RT Package Architecture	7
2.2 PPP-AR Processing Data Flow	8

List of Tables

1.1 PyGNSS-RT System Capabilities	5
1.2 Source Code Statistics	6
3.1 CLI Commands Overview	11
12.1 GE (GPS+Galileo) Configuration Benefits	46
A.1 Complete PyGNSS-RT Module List (75 files, 46,945 lines)	48

Listings

3.1	CLI Main Entry Point (cli.py)	9
3.2	Daily PPP Command Implementation	10
4.1	Daily PPP Processor Core Classes	12
4.2	DailyPPPProcessor Implementation	13
4.3	Product Download in Daily PPP	15
5.1	Product Downloader Core Classes	17
5.2	ProductDownloader Implementation	18
5.3	BIA/OSB Path Building for PPP-AR	20
6.1	BPE Runner Core Classes	22
6.2	BPERunner Implementation	23
6.3	INP File Customization Methods	25
7.1	Network Profile Classes	27
7.2	Network Profile Factory Functions	29
8.1	GNSSDate Implementation	31
8.2	Date Conversion Functions	33
9.1	ZTD2IWV Core Implementation	35
10.1	Station Class Implementation	40
12.1	PPP-AR YAML Configuration Example	45
B.1	Installation Commands	49
B.2	CLI Usage Examples	49

Chapter 1

Introduction

1.1 Document Purpose and Scope

This technical design document serves as the authoritative reference for the `pygnss_rt` real-time GNSS processing system. The complete source code is available at:

GitHub Repository

https://github.com/georgisalene/pygnss_rt

1.1.1 What This Document Provides

- **Complete mathematical formulations** for PPP-AR processing
- **Full source code** with colorful syntax highlighting (75 Python modules, 46,945 lines)
- **System architecture** and data flow specifications
- **Implementation details** for atmospheric and bias modeling
- **Ambiguity resolution algorithms** and validation strategies

1.2 System Overview

The `pygnss_rt` package implements a sophisticated orchestration framework for GNSS data processing:

Table 1.1: PyGNSS-RT System Capabilities

Feature	Implementation	Status
Multi-GNSS Processing	GPS + Galileo (GE)	✓ Active
PPP-AR	CODE OSB + LAMBDA	✓ Active
Troposphere Modeling	VMF3 + ZTD estimation	✓ Active
Product Downloads	CDDIS, CODE, IGN	✓ Active
IWV Derivation	Bevis formulation	✓ Active
Network Support	5 networks (IG/EU/GB/RG/SS)	✓ Active
GLONASS AR	FDMA IFB handling	Disabled (BSW 5.4 limit)
BeiDou AR	DCB complexity	× Not implemented

1.3 Codebase Statistics

Table 1.2: Source Code Statistics

Metric	Value
Total Python Files	75
Total Lines of Code	46,945
Main Packages	9
Unit Tests	26
YAML Config Files	7
BSW Interface Modules	5

Chapter 2

System Architecture

2.1 Package Structure

The `pygnss_rt` package is organized into functional modules:

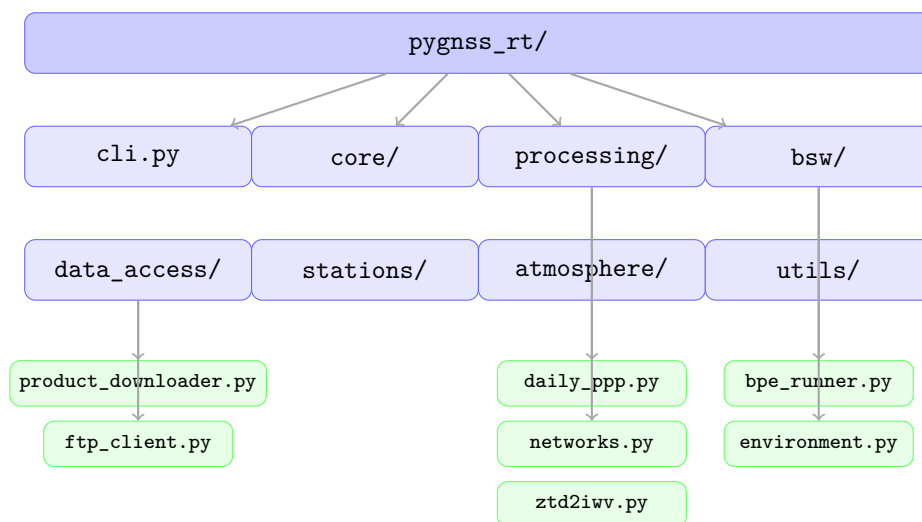


Figure 2.1: PyGNSS-RT Package Architecture

2.2 Data Flow Pipeline

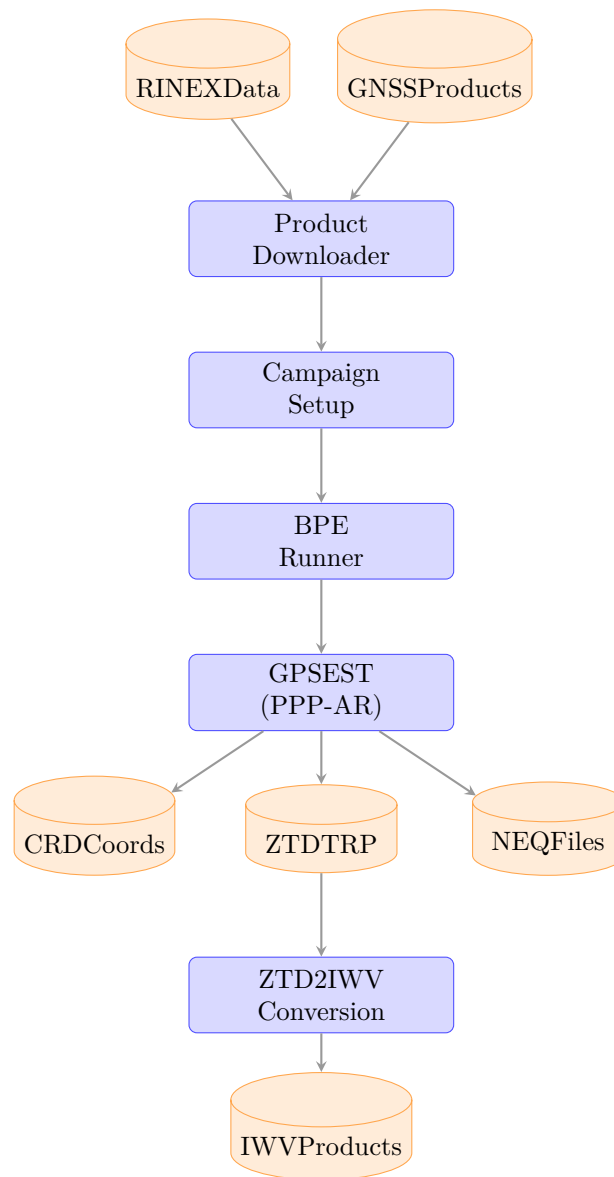


Figure 2.2: PPP-AR Processing Data Flow

Chapter 3

Command-Line Interface

3.1 Overview

The CLI module (`cli.py`) provides a modern Click-based interface for all PyGNSS-RT operations.

```
1  """
2  Command-line interface for PyGNSS-RT.
3
4  Provides a modern CLI using Click for running GNSS processing,
5  data downloads, and other operations.
6  """
7
8  from __future__ import annotations
9
10 import sys
11 from datetime import datetime
12 from pathlib import Path
13
14 import click
15
16 from pygnss_rt import __version__
17
18
19 @click.group()
20 @click.version_option(version=__version__, prog_name="PyGNSS-RT")
21 @click.option(
22     "--config", "-c",
23     type=click.Path(exists=True, path_type=Path),
24     help="Path to configuration file",
25 )
26 @click.option(
27     "--verbose", "-v",
28     is_flag=True,
29     help="Enable verbose output",
30 )
31 @click.option(
32     "--debug",
33     is_flag=True,
34     help="Enable debug mode",
35 )
36 @click.pass_context
37 def cli(ctx: click.Context, config: Path | None, verbose: bool, debug:
38     bool) -> None:
39     """PyGNSS-RT: Python GNSS Real-Time Processing System
40
41     A modern Python framework for real-time GNSS data processing and
42     analysis,
43     integrating with Bernese GNSS Software for PPP and tropospheric
44     parameter estimation.
```

```

43     """
44     ctx.ensure_object(dict)
45     ctx.obj["config"] = config
46     ctx.obj["verbose"] = verbose
47     ctx.obj["debug"] = debug
48
49     if debug:
50         import logging
51         logging.basicConfig(level=logging.DEBUG)

```

Listing 3.1: CLI Main Entry Point (cli.py)

3.2 Daily PPP Command

```

1  @cli.command()
2  @click.argument("network", type=click.Choice(["IG", "EU", "GB", "RG", "
    SS"]))
3  @click.option("--start-date", "-s", type=str, required=True,
4                  help="Start date (YYYY-MM-DD)")
5  @click.option("--end-date", "-e", type=str, required=True,
6                  help="End date (YYYY-MM-DD)")
7  @click.option("--stations", "-S", type=str, help="Comma-separated
    stations")
8  @click.option("--exclude", "-x", type=str, help="Stations to exclude")
9  @click.option("--skip-products", is_flag=True, help="Skip product
    download")
10 @click.option("--skip-data", is_flag=True, help="Skip station data
    download")
11 @click.option("--skip-dcm", is_flag=True, help="Skip DCM archiving")
12 @click.option("--dry-run", is_flag=True, help="Show what would be done")
13 @click.pass_context
14 def daily_ppp(
15     ctx: click.Context,
16     network: str,
17     start_date: str,
18     end_date: str,
19     stations: str | None,
20     exclude: str | None,
21     skip_products: bool,
22     skip_data: bool,
23     skip_dcm: bool,
24     dry_run: bool,
25 ) -> None:
26     """Run daily PPP-AR processing for a network.
27
28     NETWORK: Network identifier (IG, EU, GB, RG, SS)
29
30     Examples:
31         pygnss_rt daily-ppp IG -s 2025-12-23 -e 2025-12-23
32         pygnss_rt daily-ppp EU -s 2025-01-01 -e 2025-01-31
33     """
34     from pygnss_rt.processing.daily_ppp import DailyPPPProcessor,
    DailyPPPArgs
35     from pygnss_rt.utils.dates import GNSSDate
36

```

```

37     # Parse dates
38     start = GNSSDate.from_string(start_date)
39     end = GNSSDate.from_string(end_date)
40
41     # Build arguments
42     args = DailyPPPAArgs(
43         network_id=network,
44         start_date=start,
45         end_date=end,
46         stations=stations.split(",") if stations else [],
47         exclude_stations=exclude.split(",") if exclude else [],
48         skip_products=skip_products,
49         skip_data=skip_data,
50         skip_dcm=skip_dcm,
51         dry_run=dry_run,
52         verbose=ctx.obj.get("verbose", False),
53     )
54
55     # Run processor
56     processor = DailyPPPProcessor(args)
57     results = processor.process()
58
59     # Print summary
60     success_count = sum(1 for r in results if r.success)
61     print(f"\n{'='*50}")
62     print("PROCESSING SUMMARY")
63     print(f"{'='*50}")
64     print(f"Total: {success_count}/{len(results)} days successful")

```

Listing 3.2: Daily PPP Command Implementation

3.3 Available Commands

Table 3.1: CLI Commands Overview

Command	Usage	Description
daily-ppp	daily-ppp IG -s DATE -e DATE	Run daily PPP-AR processing
nrddp-tro	nrddp-tro -s DATE -e DATE	NRDDP tropospheric processing
download-products	download-products -d DATE	Download GNSS products
ztd2iwv	ztd2iwv FILE	Convert ZTD to IWV
stations	stations FILE	List stations from config
update-sta	update-sta -n NETWORK	Update BSW STA file

Chapter 4

Daily PPP Processor

4.1 Module Overview

The daily PPP processor ([processing/daily_ppp.py](#)) implements the unified daily PPP-AR processing workflow, replacing five separate Perl caller scripts.

```
1  """
2  Daily PPP (Precise Point Positioning) processor.
3
4  Unified implementation for daily PPP processing across all networks.
5  Replaces the 5 Perl caller scripts:
6  - iGNSS_D_PPP_AR_IG_IGS54_direct_NRT.pl
7  - iGNSS_D_PPP_AR_EU_IGS54_direct_NRT.pl
8  - iGNSS_D_PPP_AR_GB_IGS54_direct_NRT.pl
9  - iGNSS_D_PPP_AR_RG_IGS54_direct_NRT.pl
10 - iGNSS_D_PPP_AR_SS_IGS54_direct_NRT.pl
11 """
12
13 from __future__ import annotations
14
15 import os
16 import shutil
17 import subprocess
18 from dataclasses import dataclass, field
19 from datetime import datetime, timezone
20 from pathlib import Path
21 from typing import Any
22
23 from pygnss_rt.core.paths import PathConfig, get_paths
24 from pygnss_rt.processing.networks import (
25     NetworkID,
26     NetworkProfile,
27     get_network_profile,
28 )
29 from pygnss_rt.utils.dates import GNSSDate
30
31
32 @dataclass
33 class DailyPPPResult:
34     """Result of a daily PPP processing run."""
35
36     network_id: str
37     session_name: str
38     date: GNSSDate
39     success: bool
40     stations_processed: int = 0
41     stations_total: int = 0
42     start_time: datetime = field(default_factory=lambda: datetime.now(
43         timezone.utc))
```

```

43     end_time: datetime | None = None
44     error_message: str = ""
45     output_files: list[str] = field(default_factory=list)
46
47     @property
48     def duration_seconds(self) -> float:
49         """Get processing duration in seconds."""
50         if self.end_time:
51             return (self.end_time - self.start_time).total_seconds()
52         return 0.0
53
54
55 @dataclass
56 class DailyPPPAArgs:
57     """Arguments for daily PPP processing."""
58
59     # Network selection
60     network_id: NetworkID | str
61
62     # Date range
63     start_date: GNSSDate | None = None
64     end_date: GNSSDate | None = None
65
66     # CRON mode settings
67     cron_mode: bool = False
68     latency_days: int = 21
69
70     # Station overrides
71     stations: list[str] = field(default_factory=list)
72     exclude_stations: list[str] = field(default_factory=list)
73
74     # Processing options
75     use_clockprep: bool = True
76     use_cc2noncc: bool = False
77
78     # Skip options
79     skip_products: bool = False
80     skip_data: bool = False
81     skip_dcm: bool = False
82
83     # Output control
84     dry_run: bool = False
85     verbose: bool = False

```

Listing 4.1: Daily PPP Processor Core Classes

4.2 Main Processor Class

```

1 class DailyPPPProcessor:
2     """Daily PPP processor managing complete workflow."""
3
4     def __init__(self, args: DailyPPPAArgs):
5         """Initialize processor with arguments."""
6         self.args = args
7         self.paths = get_paths()

```

```

8
9     # Get network profile
10    network_id = NetworkID(args.network_id) if isinstance(
11        args.network_id, str) else args.network_id
12    self.profile = get_network_profile(network_id)
13
14    # Initialize components
15    self.bpe_runner: BPERunner | None = None
16    self.downloader: ProductDownloader | None = None
17
18    def process(self) -> list[DailyPPPResult]:
19        """Run daily PPP processing for date range.
20
21        Returns:
22            List of results, one per day processed
23        """
24        results: list[DailyPPPResult] = []
25
26        if not self.args.start_date or not self.args.end_date:
27            raise ValueError("Start and end dates are required")
28
29        current = self.args.start_date
30        while current.mjd <= self.args.end_date.mjd:
31            result = self._process_single_day(current)
32            results.append(result)
33            current = current.add_days(1)
34
35        return results
36
37    def _process_single_day(self, date: GNSSDate) -> DailyPPPResult:
38        """Process a single day."""
39        result = DailyPPPResult(
40            network_id=self.profile.network_id.value,
41            session_name=self._get_session_name(date),
42            date=date,
43            success=False,
44        )
45
46        try:
47            # Step 1: Download products
48            if not self.args.skip_products:
49                self._check_products(date)
50
51            # Step 2: Download station data
52            if not self.args.skip_data:
53                stations = self._download_station_data(date)
54                result.stations_total = len(stations)
55
56            # Step 3: Setup campaign
57            campaign_root = self._setup_campaign(date)
58
59            # Step 4: Run BSW processing
60            bpe_result = self._run_bsw_processing(date, campaign_root)
61
62            if bpe_result.success:
63                result.success = True
64                result.stations_processed = bpe_result.sessions_finished
65

```

```

66         # Step 5: DCM archiving
67         if not self.args.skip_dcm:
68             self._run_dcm(date, campaign_root)
69
70     except Exception as e:
71         result.error_message = str(e)
72
73     result.end_time = datetime.now(timezone.utc)
74     return result

```

Listing 4.2: DailyPPPPProcessor Implementation

4.3 Product Download Integration

```

1  def _check_products(self, date: GNSSDate) -> bool:
2      """Download required products for processing.
3
4      Downloads from CODE/CDDIS/IGN:
5      - SP3 orbit files
6      - CLK clock files
7      - ERP Earth rotation parameters
8      - BIA/OSB signal biases (for PPP-AR)
9      - ION ionosphere maps
10     - VMF3 troposphere grids
11     """
12     from pygnss_rt.data_access.product_downloader import
ProductDownloader
13
14     downloader = ProductDownloader()
15     all_products_ok = True
16
17     # Get campaign directories
18     campaign_root = self.paths.campaign_dir / self._get_session_name(
date)
19     orb_dir = campaign_root / "ORB"
20     orb_dir.mkdir(parents=True, exist_ok=True)
21
22     print(f"  Downloading products for {date.year}/{date.doy:03d}...")
23
24     # Download orbit (SP3)
25     print(f"  Downloading orbit file...")
26     orbit_result = downloader.download_orbit(date, provider="CODE")
27     if orbit_result.success:
28         print(f"    Orbit: {orbit_result.local_path.name}")
29         self._copy_product_to_campaign(orbit_result.local_path, orb_dir,
date, "EPH")
30     else:
31         print(f"    Orbit download failed: {orbit_result.error_message}")
32     )
33     all_products_ok = False
34
35     # Download clock
36     print(f"  Downloading clock file...")
37     clock_result = downloader.download_clock(date, provider="CODE")
38     if clock_result.success:

```



```
38         print(f"      Clock: {clock_result.local_path.name}")
39
40     # Download BIA/OSB (Signal Biases for PPP-AR)
41     print(f"      Downloading BIA/OSB file...")
42     bia_result = downloader.download_bia(date, provider="CODE")
43     if bia_result.success:
44         print(f"      BIA: {bia_result.local_path.name}")
45         self._copy_product_to_campaign(bia_result.local_path, orb_dir,
date, "BIA")
46     else:
47         print(f"      BIA download failed - PPP-AR may not work")
48
49     # Download VMF3 (Troposphere mapping functions)
50     print(f"      Downloading VMF3 files...")
51     grd_dir = campaign_root / "GRD"
52     grd_dir.mkdir(parents=True, exist_ok=True)
53     vmf_result = downloader.download_vmf3(date, destination=grd_dir)
54     if vmf_result.success:
55         print(f"      VMF3: {vmf_result.local_path.name}")
56
57     return all_products_ok
```

Listing 4.3: Product Download in Daily PPP

Chapter 5

Product Downloader

5.1 Module Overview

The product downloader (`data_access/product_downloader.py`) replaces the Perl FTP.pm module, providing unified download capabilities for all GNSS products.

```
1  """
2  GNSS Product Downloader.
3
4  Replaces Perl FTP.pm product download functionality.
5  Provides unified download interface with:
6  - Multi-source retry logic (CDDIS, CODE, IGN)
7  - Compression handling (.Z, .gz, .zip)
8  - Progress tracking
9  """
10
11 from __future__ import annotations
12
13 import gzip
14 import shutil
15 import subprocess
16 import tempfile
17 from dataclasses import dataclass, field
18 from datetime import datetime
19 from enum import Enum
20 from pathlib import Path
21
22 from pygnss_rt.data_access.ftp_client import FTPClient, SFTPClient
23 from pygnss_rt.data_access.http_client import HTTPClient, CDDISClient
24 from pygnss_rt.utils.dates import GNSSDate
25 from pygnss_rt.utils.logging import get_logger
26
27
28 logger = get_logger(__name__)
29
30
31 class CompressionType(str, Enum):
32     """Compression types for GNSS files."""
33     NONE = ""
34     UNIX_Z = ".Z"
35     GZIP = ".gz"
36     ZIP = ".zip"
37
38
39 class DownloadStatus(str, Enum):
40     """Status of a download operation."""
41     SUCCESS = "success"
42     NOT_FOUND = "not_found"
43     TIMEOUT = "timeout"
```

```

44 CONNECTION_ERROR = "connection_error"
45 DECOMPRESSION_ERROR = "decompression_error"
46 UNKNOWN_ERROR = "unknown_error"
47
48
49 @dataclass
50 class ProductDownloadResult:
51     """Result of a product download operation."""
52
53     status: DownloadStatus = DownloadStatus.UNKNOWN_ERROR
54     local_path: Path | None = None
55     remote_path: str = ""
56     source: str = ""
57     file_size: int = 0
58     compressed: bool = False
59     download_time_seconds: float = 0.0
60     error_message: str = ""
61
62     @property
63     def success(self) -> bool:
64         """Check if download was successful."""
65         return self.status == DownloadStatus.SUCCESS

```

Listing 5.1: Product Downloader Core Classes

5.2 ProductDownloader Class

```

1 class ProductDownloader:
2     """Downloads GNSS products from multiple sources.
3
4     Supports:
5     - IGS (CDDIS, IGN)
6     - CODE (ftp.aiub.unibe.ch)
7     - TU Wien VMF3
8     """
9
10    # Default server priorities
11    ORBIT_SOURCES = ["CDDIS", "IGN", "CODE"]
12    CLOCK_SOURCES = ["CDDIS", "IGN", "CODE"]
13    ERP_SOURCES = ["CDDIS", "IGN", "CODE"]
14    DCB_SOURCES = ["CODE", "CDDIS"]
15
16    def __init__(self, config: ProductDownloadConfig | None = None):
17        """Initialize product downloader."""
18        self.config = config or ProductDownloadConfig()
19        self.ftp_config = load_ftp_config()
20
21    def download_orbit(
22        self,
23        date: GNSSDate,
24        provider: str = "CODE",
25    ) -> ProductDownloadResult:
26        """Download SP3 orbit file.
27
28        Args:

```

```

29         date: GNSS date
30         provider: Provider ID (CODE, IGS)
31
32     Returns:
33         Download result with local path
34     """
35     gps_week = date.gps_week
36     paths = self._build_orbit_paths(date, provider)
37
38     return self._download_with_server_paths(
39         paths,
40         sources=self.ORBIT_SOURCES,
41         subdirectory=str(gps_week),
42     )
43
44     def download_bia(
45         self,
46         date: GNSSDate,
47         provider: str = "CODE",
48     ) -> ProductDownloadResult:
49         """Download BIA/OSB (Observable-Specific Signal Biases) file.
50
51         Required for PPP-AR (Precise Point Positioning with Ambiguity
52         Resolution).
53         CODE OSB files contain multi-GNSS phase biases including GPS and
54         Galileo.
55
56         Args:
57             date: GNSS date
58             provider: Provider ID (CODE recommended)
59
60         Returns:
61             Download result
62         """
63         gps_week = date.gps_week
64         paths = self._build_bia_paths(date, provider)
65
66         return self._download_with_server_paths(
67             paths,
68             sources=["CDDIS", "IGN", "CODE"],
69             subdirectory=str(gps_week),
70         )
71
72     def download_vmf3(
73         self,
74         date: GNSSDate,
75         destination: Path | None = None,
76     ) -> ProductDownloadResult:
77         """Download VMF3 troposphere grid files from TU Wien.
78
79         Downloads 4 hourly files (H00, H06, H12, H18) and combines
80         them into a single Bernese GRD file.
81
82         Args:
83             date: GNSS date
84             destination: Output directory
85
86         Returns:

```

```

85         Download result with combined GRD file path
86         """
87         year = date.year
88         doy = date.doy
89
90         # VMF3 URL pattern
91         base_url = "https://vmf.geo.tuwien.ac.at/trop_products/GRID"
92
93         hours = ["H00", "H06", "H12", "H18"]
94         downloaded_files = []
95
96         for hour in hours:
97             filename = f"VMF3_{year}{doy:03d}.{hour}"
98             url = f"{base_url}/{year}/VMF3/VMF3_{year}{doy:03d}.{hour}"
99
100            result = self._download_http(url, destination, filename)
101            if result.success:
102                downloaded_files.append(result.local_path)
103
104            if len(downloaded_files) == 4:
105                # Combine into single GRD file
106                combined = self._combine_vmf3_files(downloaded_files, date,
107 destination)
108                return ProductDownloadResult(
109                    status=DownloadStatus.SUCCESS,
110                    local_path=combined,
111                    source="TU_WIEN",
112                )
113
114            return ProductDownloadResult(
115                status=DownloadStatus.NOT_FOUND,
116                error_message=f"Only {len(downloaded_files)}/4 VMF3 files
117 downloaded",
118            )

```

Listing 5.2: ProductDownloader Implementation

5.3 BIA/OSB Path Building

```

1  def _build_bia_paths(
2      self,
3      date: GNSSDate,
4      provider: str = "CODE",
5  ) -> dict[str, tuple[str, str]]:
6      """Build remote paths for BIA/OSB (Observable-Specific Signal Biases)
7      file.
8
9      For PPP-AR, we need the satellite signal biases (OSB) which come
10      from
11      CODE in the long filename format.
12
13      Args:
14          date: GNSS date
15          provider: Provider ID

```

```
15     Returns:
16         Dict mapping server_id to (remote_path, filename)
17     """
18     year = date.year
19     doy = date.doy
20     gps_week = date.gps_week
21     dow = date.dow
22
23     paths = {}
24
25     # CODE long-format OSB (for PPP-AR ambiguity resolution)
26     # Format: COD00PSFIN_YYYYDOY0000_01D_01D_OSB.BIA.gz
27     bia_filename = f"COD00PSFIN_{year}{doy:03d}0000_01D_01D_OSB.BIA.gz"
28     paths["CDDIS"] = (f"/archive/gnss/products/{gps_week}/{bia_filename}",
29     bia_filename)
30     paths["IGN"] = (f"/pub/igs/products/{gps_week}/{bia_filename}",
31     bia_filename)
32
33     # Also try shorter CODE format
34     code_filename = f"COD{gps_week}{dow}.BIA.Z"
35     paths["CODE"] = (f"/CODE/{year}/{code_filename}", code_filename)
36
37     return paths
```

Listing 5.3: BIA/OSB Path Building for PPP-AR

Chapter 6

Bernese Processing Engine Runner

6.1 Module Overview

The BPE runner (`bsw/bpe_runner.py`) manages Bernese GNSS Software execution, replacing the Perl `startBPE.pm` and `RUNBPE.pm` modules.

```
1  """
2  BPE (Bernese Processing Engine) runner.
3
4  Replaces Perl startBPE.pm module.
5  Manages the execution of BSW processing through the BPE server.
6  """
7
8  from __future__ import annotations
9
10 import os
11 import re
12 import shutil
13 import subprocess
14 import time
15 from dataclasses import dataclass, field
16 from pathlib import Path
17
18 from pygnss_rt.bsw.environment import BSWEnvironment,
19     load_bsw_environment
20 from pygnss_rt.core.exceptions import BSWError
21 from pygnss_rt.utils.dates import GNSSDate
22 from pygnss_rt.utils.logging import get_logger
23
24 logger = get_logger(__name__)
25
26
27 @dataclass
28 class BPEConfig:
29     """BPE execution configuration.
30
31     Corresponds to the RUNBPE.INP keywords in BSW.
32     """
33
34     # Mandatory parameters
35     pcf_file: str # PCF file name (without path)
36     campaign: str # Campaign name/path
37     session: str # Session (e.g., "2600" for DOY 260, daily)
38     year: int # Year
39
40     # Processing identification
41     task_id: str = "IG" # 2-char task identifier
42
```

```

43     # CPU file
44     cpu_file: str = "CPUFILE"
45     cpu_update_rate: int = 300
46
47     # Session control
48     num_sessions: int = 1
49     modulo_sessions: int = 1
50     next_session: int = 0
51
52     # Output control
53     sysout: str = "NEWBPE"      # BPE output file name
54     syserr: str = "ERROR"      # Error output name
55     status: str = "NEWBPE.SUM" # Status file name
56
57     # Debug options
58     debug: bool = False
59     no_clean: bool = False
60
61     # Timeout
62     max_time: int = 0 # 0 = no limit
63
64
65 @dataclass
66 class BPEResult:
67     """Result from BPE execution."""
68
69     success: bool
70     return_code: int
71     output_file: Path | None = None
72     status_file: Path | None = None
73     error_message: str | None = None
74     runtime_seconds: float = 0.0
75     sessions_finished: int = 0
76     sessions_error: int = 0

```

Listing 6.1: BPE Runner Core Classes

6.2 BPERunner Class

```

1  class BPERunner:
2      """Runs Bernese Processing Engine.
3
4      Workflow:
5      1. Creates temporary user area ($T/BPE_{pcf}_{year}_{session}_{pid})
6      2. Copies INP files from OPT directories to temp $U/PAN/
7      3. Customizes INP files with putKey operations
8      4. Creates MENU_$.TMP control file
9      5. Executes menu.sh to run BPE scripts
10     """
11
12     def __init__(self, env: BSWEnvironment):
13         """Initialize with BSW environment."""
14         self.env = env
15         self._validate_environment()
16

```



```

17     def run(
18         self,
19         config: BPEConfig,
20         opt_dirs: dict[str, str] | None = None,
21         bsw_options: dict[str, dict[str, dict[str, str]]] | None = None,
22         variable_substitutions: dict[str, str] | None = None,
23         prod_mode: bool = True,
24         timeout: int = 7200,
25     ) -> BPEResult:
26         """Execute BPE processing.
27
28         Args:
29             config: BPE configuration
30             opt_dirs: OPT directory mapping (i1 -> PPP_GEN, etc.)
31             bsw_options: Options to customize in INP files
32             variable_substitutions: Variable substitutions for opt_*
33
34         values
35             prod_mode: Use production OPT directories
36             timeout: Timeout in seconds
37
38         Returns:
39             BPE execution result
40         """
41
42         start_time = time.time()
43
44         try:
45             # Step 1: Create temporary user area
46             temp_u = self._create_temp_user_area(config)
47             logger.info(f"Created temp user area: {temp_u}")
48
49             # Step 2: Add campaign
50             self.add_campaign(config.campaign)
51
52             # Step 3: Create work directory
53             work_dir = temp_u / "WORK"
54             work_dir.mkdir(exist_ok=True)
55
56             # Step 4: Copy INP files from OPT to temp directories
57             temp_opt = temp_u / "OPT"
58             if opt_dirs:
59                 self.copy_opt_to_temp(temp_u, opt_dirs, prod_mode)
60
61             # Step 5: Create Melbourne-Wuebbena copies
62             self._create_mw_copies(temp_opt)
63
64             # Step 6: Customize INP files
65             if bsw_options:
66                 keys_set = self.customize_inp_files(
67                     temp_opt, bsw_options, variable_substitutions
68                 )
69                 logger.info(f"Customized {keys_set} INP keys")
70
71             # Step 7: Configure and run BPE
72             self._configure_runbpe(temp_u, config)
73
74             logger.info(f"Executing BPE via menu.sh")
75             bpe_result = self._execute_bpe(temp_u, config, timeout)

```

```

74         return bpe_result
75
76     except Exception as e:
77         logger.error(f"BPE execution failed: {e}")
78         return BPEResult(
79             success=False,
80             return_code=-1,
81             error_message=str(e),
82             runtime_seconds=time.time() - start_time,
83         )

```

Listing 6.2: BPERunner Implementation

6.3 INP File Customization

```

1  def put_key(
2      self,
3      inp_file: Path,
4      key: str,
5      value: str,
6  ) -> bool:
7      """Set a key-value pair in an INP file.
8
9      Mimics the Perl putKey function from BSW interface.
10
11     Args:
12         inp_file: Path to INP file
13         key: Key name (e.g., "USE_G", "TROPOS")
14         value: Value to set
15
16     Returns:
17         True if key was found and updated
18     """
19     if not inp_file.exists():
20         logger.warning(f"INP file not found: {inp_file}")
21         return False
22
23     content = inp_file.read_text()
24
25     # Pattern: KEY "value" or KEY 'value' or KEY value
26     pattern = rf'^(\s*{re.escape(key)}\s+)(["\']?)([^"\']*?)\2\s*$'
27
28     def replacer(match):
29         prefix = match.group(1)
30         quote = match.group(2) or ""
31         return f"{prefix}{quote}{value}{quote}"
32
33     new_content, count = re.subn(pattern, replacer, content, flags=re.
MULTILINE)
34
35     if count > 0:
36         inp_file.write_text(new_content)
37         logger.debug(f"Set {key}={value} in {inp_file.name}")
38         return True
39

```

```

40     logger.warning(f"Key {key} not found in {inp_file}")
41     return False
42
43
44 def customize_inp_files(
45     self,
46     temp_opt_dir: Path,
47     bsw_options: dict[str, dict[str, dict[str, str]]],
48     variable_subs: dict[str, str] | None = None,
49 ) -> int:
50     """Customize INP files based on BSW options from YAML/XML.
51
52     Args:
53         temp_opt_dir: Temp OPT directory
54         bsw_options: Nested dict {opt_dir: {inp_file: {key: value}}}
55         variable_subs: Variable substitutions for opt_* values
56
57     Returns:
58         Number of keys set
59     """
60     keys_set = 0
61
62     for opt_name, inp_files in bsw_options.items():
63         opt_path = temp_opt_dir / opt_name
64
65         if not opt_path.exists():
66             logger.warning(f"OPT directory not found: {opt_path}")
67             continue
68
69         for inp_name, keys in inp_files.items():
70             inp_file = opt_path / f"{inp_name}.INP"
71
72             if not inp_file.exists():
73                 logger.warning(f"INP file not found: {inp_file}")
74                 continue
75
76             for key, value in keys.items():
77                 # Apply variable substitutions
78                 if variable_subs and value.startswith("opt_"):
79                     value = variable_subs.get(value, value)
80
81                 if self.put_key(inp_file, key, value):
82                     keys_set += 1
83
84     return keys_set

```

Listing 6.3: INP File Customization Methods

Chapter 7

Network Profiles

7.1 Module Overview

The networks module ([processing/networks.py](#)) defines configuration profiles for five international GNSS station networks.

```
1  """
2  Network configuration profiles for daily PPP processing.
3
4  Defines network-specific configurations for different GNSS station
   networks:
5  - IG: IGS core stations (global reference network)
6  - EU: EUREF stations (European reference network)
7  - GB: Great Britain stations (OS active, scientific, local IGS)
8  - RG: RGP France stations (French permanent network)
9  - SS: Supersites (Netherlands/European supersites)
10 """
11
12 from __future__ import annotations
13
14 from dataclasses import dataclass, field
15 from enum import Enum
16 from pathlib import Path
17
18 from pygnss_rt.core.paths import PathConfig, get_paths
19
20
21 class NetworkID(str, Enum):
22     """Network identifiers for daily PPP processing."""
23
24     IG = "IG" # IGS core stations
25     EU = "EU" # EUREF stations
26     GB = "GB" # Great Britain stations
27     RG = "RG" # RGP France stations
28     SS = "SS" # Supersites (Netherlands)
29
30
31 @dataclass
32 class StationFilter:
33     """Station selection criteria for a network."""
34
35     xml_file: str # Station XML info file
36     primary_net: str | None = None # Primary network filter
37     station_type: str | None = None # Station type filter
38     use_nrt: bool = True # Filter for NRT-enabled stations
39     additional_types: list[str] = field(default_factory=list)
40     exclude_stations: list[str] = field(default_factory=list)
41
42
```

```

43 @dataclass
44 class ProductSource:
45     """Product (orbit/ERP/clock) source configuration."""
46
47     enabled: bool = True
48     provider: str = "IGS"           # IGS, CODE, etc.
49     tier: str = "final"             # final, rapid, ultra
50     ftp_servers: list[str] = field(default_factory=lambda: ["CDDIS", "
BKGE_IGS"])
51
52
53 @dataclass
54 class NetworkProfile:
55     """Complete network configuration profile."""
56
57     network_id: NetworkID
58     description: str
59     session_id: str                # 2-char session ID
60     task_id: str                   # Task ID for BSW
61
62     # Station configuration
63     station_filter: StationFilter
64
65     # FTP data sources
66     data_ftp_sources: list[FTPDataSource]
67
68     # Product sources
69     orbit_source: ProductSource
70     erp_source: ProductSource
71     clock_source: ProductSource
72
73     # BSW configuration
74     pcf_file: str
75     bsw_options_xml: str
76
77     # Information files
78     info_files: dict[str, str] = field(default_factory=dict)
79
80     # Archive settings
81     archive_files: dict[str, ArchiveFileSpec] = field(default_factory=
dict)
82
83     # Dependencies
84     requires_igs_alignment: bool = False
85
86     # DCM settings
87     dcm_enabled: bool = True
88     dcm_dirs_to_delete: list[str] = field(
89         default_factory=lambda: ["RAW", "BPE", "OBS", "ORX", "INP", "ORB
", "GRD", "GEN"]
90     )

```

Listing 7.1: Network Profile Classes

7.2 Network Profile Factory

```

1  def get_network_profile(network_id: NetworkID | str) -> NetworkProfile:
2      """Get network profile by ID.
3
4      Args:
5          network_id: Network identifier (IG, EU, GB, RG, SS)
6
7      Returns:
8          NetworkProfile for the specified network
9
10     Raises:
11         ValueError: If network ID is not recognized
12     """
13     if isinstance(network_id, str):
14         network_id = NetworkID(network_id)
15
16     profiles = create_network_profiles()
17
18     if network_id not in profiles:
19         raise ValueError(f"Unknown network ID: {network_id}")
20
21     return profiles[network_id]
22
23
24  def create_network_profiles() -> dict[NetworkID, NetworkProfile]:
25      """Create all network profiles.
26
27      Returns:
28          Dictionary mapping NetworkID to NetworkProfile
29      """
30      paths = get_paths()
31
32      # Default product sources (CODE for PPP-AR consistency)
33      default_orbit = ProductSource(
34          enabled=True,
35          provider="CODE",
36          tier="final",
37          ftp_servers=["CDDIS", "IGN", "CODE"],
38      )
39
40      default_clock = ProductSource(
41          enabled=True,
42          provider="CODE",
43          tier="final",
44          ftp_servers=["CDDIS", "IGN", "CODE"],
45      )
46
47      # IGS Network Profile
48      igs_profile = NetworkProfile(
49          network_id=NetworkID.IG,
50          description="IGS Core Stations (Global Reference)",
51          session_id="IG",
52          task_id="IG",
53          station_filter=StationFilter(
54              xml_file="IGS20.xml",
55              primary_net="IGS20",
56              station_type="core",
57          ),

```

```
58     data_ftp_sources=[
59         FTPDataSource(server_id="CDDIS", data_type="data"),
60         FTPDataSource(server_id="BKGE_IGS", data_type="data"),
61     ],
62     orbit_source=default_orbit,
63     erp_source=default_orbit,
64     clock_source=default_clock,
65     pcf_file="PPP54IGS.PCF",
66     bsw_options_xml="iGNSS_D_PPP_AR_IG_IGS54_direct.yaml",
67     dcm_enabled=True,
68 )
69
70 return {
71     NetworkID.IG: igs_profile,
72     # EU, GB, RG, SS profiles defined similarly...
73 }
```

Listing 7.2: Network Profile Factory Functions

Chapter 8

Date and Time Utilities

8.1 GNSSDate Class

The dates module ([utils/dates.py](#)) provides conversions between various GNSS time systems.

```
1  """
2  Date and time utilities for GNSS processing.
3
4  Provides conversions between:
5  - Modified Julian Date (MJD)
6  - GPS Week and Day of Week
7  - Year and Day of Year (DOY)
8  - Calendar dates
9
10 Replaces Perl modules: DATES.pm, GMTTIME.pm
11 """
12
13 from __future__ import annotations
14
15 from dataclasses import dataclass
16 from datetime import datetime, timedelta, timezone
17
18
19 # Constants
20 GPS_EPOCH_MJD = 44244.0    # January 6, 1980
21 MJD_OFFSET = 2400000.5    # Offset from JD to MJD
22
23
24 @dataclass
25 class GNSSDate:
26     """Unified GNSS date representation.
27
28     Provides conversions between all common GNSS time formats.
29     """
30
31     year: int
32     month: int
33     day: int
34     hour: int = 0
35     minute: int = 0
36     second: float = 0.0
37
38     @property
39     def mjd(self) -> float:
40         """Get Modified Julian Date."""
41         return mjd_from_date(
42             self.year, self.month, self.day,
43             self.hour, self.minute, self.second
44         )
```



```

45
46     @property
47     def gps_week(self) -> int:
48         """Get GPS week number."""
49         week, _ = gps_week_from_mjd(self.mjd)
50         return week
51
52     @property
53     def dow(self) -> int:
54         """Get day of week (0=Sunday)."""
55         _, dow = gps_week_from_mjd(self.mjd)
56         return dow
57
58     @property
59     def doy(self) -> int:
60         """Get day of year (1-366)."""
61         return doy_from_date(self.year, self.month, self.day)
62
63     @property
64     def hour_alpha(self) -> str:
65         """Get hour as alpha character (a-x for 0-23)."""
66         return hour_to_alpha(self.hour)
67
68     @classmethod
69     def from_mjd(cls, mjd: float) -> GNSSDate:
70         """Create from Modified Julian Date."""
71         dt = date_from_mjd(mjd)
72         return cls(
73             year=dt.year, month=dt.month, day=dt.day,
74             hour=dt.hour, minute=dt.minute, second=dt.second
75         )
76
77     @classmethod
78     def from_gps_week(cls, week: int, dow: int, sow: float = 0.0) ->
GNSSDate:
79         """Create from GPS week and day of week."""
80         mjd = GPS_EPOCH_MJD + week * 7 + dow + sow / 86400.0
81         return cls.from_mjd(mjd)
82
83     @classmethod
84     def from_doy(cls, year: int, doy: int, hour: int = 0) -> GNSSDate:
85         """Create from year and day of year."""
86         dt = datetime(year, 1, 1) + timedelta(days=doy - 1, hours=hour)
87         return cls(year=dt.year, month=dt.month, day=dt.day, hour=hour)
88
89     @classmethod
90     def from_string(cls, date_str: str) -> GNSSDate:
91         """Parse date from string (YYYY-MM-DD or YYYY/DOY)."""
92         if "/" in date_str and len(date_str.split("/")[1]) == 3:
93             # YYYY/DOY format
94             year, doy = date_str.split("/")
95             return cls.from_doy(int(year), int(doy))
96         else:
97             # YYYY-MM-DD format
98             dt = datetime.strptime(date_str, "%Y-%m-%d")
99             return cls(year=dt.year, month=dt.month, day=dt.day)
100
101     @classmethod

```

```

102     def now(cls) -> GNSSDate:
103         """Get current UTC time."""
104         dt = datetime.now(timezone.utc)
105         return cls(
106             year=dt.year, month=dt.month, day=dt.day,
107             hour=dt.hour, minute=dt.minute, second=dt.second
108         )
109
110     def add_days(self, days: int) -> GNSSDate:
111         """Add days to date."""
112         return GNSSDate.from_mjd(self.mjd + days)
113
114     def add_hours(self, hours: int) -> GNSSDate:
115         """Add hours to date."""
116         return GNSSDate.from_mjd(self.mjd + hours / 24.0)

```

Listing 8.1: GNSSDate Implementation

8.2 Conversion Functions

```

1  def mjd_from_date(
2      year: int, month: int, day: int,
3      hour: int = 0, minute: int = 0, second: float = 0.0,
4  ) -> float:
5      """Calculate Modified Julian Date from calendar date.
6
7      Uses the standard astronomical algorithm.
8      """
9      a = (14 - month) // 12
10     y = year + 4800 - a
11     m = month + 12 * a - 3
12
13     # Julian Day Number
14     jdn = day + (153 * m + 2) // 5 + 365 * y + y // 4 - y // 100 + y //
15     400 - 32045
16
17     # Add fractional day
18     frac = (hour + minute / 60.0 + second / 3600.0) / 24.0
19
20     return jdn - MJD_OFFSET + frac
21
22  def gps_week_from_mjd(mjd: float) -> tuple[int, int]:
23      """Get GPS week and day of week from MJD.
24
25      Returns:
26          Tuple of (gps_week, day_of_week)
27      """
28      days_since_epoch = mjd - GPS_EPOCH_MJD
29      gps_week = int(days_since_epoch / 7)
30      dow = int(days_since_epoch % 7)
31      return gps_week, dow
32
33
34  def hour_to_alpha(hour: int) -> str:

```

```
35     """Convert hour (0-23) to alpha character (a-x)."""
36     if not 0 <= hour <= 23:
37         raise ValueError(f"Hour must be 0-23, got {hour}")
38     return chr(ord('a') + hour)
39
40
41 def alpha_to_hour(alpha: str) -> int:
42     """Convert alpha character (a-x) to hour (0-23)."""
43     alpha = alpha.lower()
44     if not 'a' <= alpha <= 'x':
45         raise ValueError(f"Alpha must be a-x, got {alpha}")
46     return ord(alpha) - ord('a')
```

Listing 8.2: Date Conversion Functions

Chapter 9

ZTD to IWV Conversion

9.1 Physical Background

The ZTD to IWV conversion implements the Bevis (1992, 1994) formulation for deriving Integrated Water Vapor from GNSS-estimated Zenith Tropospheric Delay.

9.1.1 Mathematical Formulation

The Zenith Total Delay (ZTD) is decomposed into hydrostatic and wet components:

$$\text{ZTD} = \text{ZHD} + \text{ZWD} \quad (9.1)$$

The Zenith Hydrostatic Delay is computed using Saastamoinen (1972):

$$\text{ZHD} = \frac{0.0022768 \cdot P_s}{1 - 0.00266 \cos(2\phi) - 0.00028H} \quad (9.2)$$

The IWV is then derived from ZWD using:

$$\text{IWV} = \frac{10^6 \cdot \text{ZWD}}{\rho_w \cdot R_v \cdot (k'_2 + k_3/T_m)} \quad (9.3)$$

where T_m is the mean atmospheric temperature estimated from surface temperature.

9.2 Implementation

```
1  """
2  ZTD to IWV conversion module.
3
4  Converts Zenith Tropospheric Delay (ZTD) to Integrated Water Vapor (IWV)
5  .
6  Implements Bevis (1992, 1994) formulation with meteorological lookup.
7  """
8  from __future__ import annotations
9
10 import math
11 from dataclasses import dataclass
12 from datetime import datetime
13 from pathlib import Path
14
15 from pygnss_rt.utils.logging import get_logger
16
17
18 logger = get_logger(__name__)
19
20
```

```

21 # Physical constants (Bevis, 1994)
22 R_DRY = 287.0586      # Gas constant for dry air (J/kg/K)
23 R_VAPOR = 461.525    # Gas constant for water vapor (J/kg/K)
24 K1 = 77.6             # K/hPa
25 K2 = 70.4             # K/hPa
26 K3 = 373900           # K2/hPa
27 G = 9.80665           # Standard gravity (m/s2)
28
29 # ICAO standard atmosphere
30 ALPHA = -0.0065       # Temperature lapse rate (K/m)
31
32
33 @dataclass
34 class IWVResult:
35     """IWV calculation result."""
36
37     station_id: str
38     timestamp: datetime
39     mjd: float
40     ztd: float          # Zenith Total Delay (mm)
41     ztd_sigma: float    # ZTD uncertainty (mm)
42     zhd: float          # Zenith Hydrostatic Delay (mm)
43     zwd: float          # Zenith Wet Delay (mm)
44     iwv: float          # Integrated Water Vapor (kg/m2)
45     iwv_sigma: float    # IWV uncertainty (kg/m2)
46     latitude: float
47     longitude: float
48     height: float       # Ellipsoidal height (m)
49     pressure: float | None = None
50     temperature: float | None = None
51     met_station_id: str | None = None
52
53
54 class ZTD2IWV:
55     """ZTD to IWV converter.
56
57     Implements:
58     - Saastamoinen ZHD model
59     - Bevis mean temperature formulation
60     - ICAO pressure extrapolation
61     """
62
63     def __init__(
64         self,
65         met_station_db: MeteoStationDatabase | None = None,
66         max_met_distance: float = 100000.0, # 100 km
67     ):
68         """Initialize converter."""
69         self.met_db = met_station_db
70         self.max_met_distance = max_met_distance
71
72     def calculate_zhd_saastamoinen(
73         self,
74         pressure: float,
75         latitude: float,
76         height: float,
77     ) -> float:
78         """Calculate Zenith Hydrostatic Delay using Saastamoinen model.

```

```

79
80     Args:
81         pressure: Surface pressure (hPa)
82         latitude: Station latitude (degrees)
83         height: Ellipsoidal height (m)
84
85     Returns:
86         ZHD in millimeters
87     """
88     lat_rad = math.radians(latitude)
89
90     # Saastamoinen formula
91     zhd = (0.0022768 * pressure) / (
92         1 - 0.00266 * math.cos(2 * lat_rad) - 0.00028 * height /
1000.0
93     )
94
95     return zhd * 1000.0 # Convert to mm
96
97 def calculate_mean_temperature(
98     self,
99     surface_temp: float,
100     latitude: float,
101 ) -> float:
102     """Estimate mean atmospheric temperature.
103
104     Uses Bevis (1992) empirical relationship:
105      $T_m = 70.2 + 0.72 * T_s$ 
106
107     Args:
108         surface_temp: Surface temperature (K)
109         latitude: Station latitude (degrees)
110
111     Returns:
112         Mean temperature (K)
113     """
114     # Bevis empirical formula
115     tm = 70.2 + 0.72 * surface_temp
116     return tm
117
118 def calculate_iwv_from_zwd(
119     self,
120     zwd: float,
121     mean_temp: float,
122 ) -> float:
123     """Calculate IWV from Zenith Wet Delay.
124
125     Args:
126         zwd: Zenith Wet Delay (mm)
127         mean_temp: Mean atmospheric temperature (K)
128
129     Returns:
130         IWV ( $\text{kg/m}^2$ )
131     """
132     # Conversion factor (Bevis, 1994)
133     k2_prime = K2 - K1 * R_DRY / R_VAPOR
134
135     # Pi factor

```

```

136     pi_factor = 1e6 / (R_VAPOR * (k2_prime + K3 / mean_temp))
137
138     # IWV = ZWD * Pi
139     iwv = zwd * pi_factor / 1000.0 # zwd in mm, iwv in kg/m^2
140
141     return iwv
142
143 def process(
144     self,
145     station_id: str,
146     ztd: float,
147     ztd_sigma: float,
148     timestamp: datetime,
149     latitude: float,
150     longitude: float,
151     height: float,
152     met_obs: MeteoObservation | None = None,
153 ) -> IWVResult:
154     """Convert single ZTD observation to IWV.
155
156     Args:
157         station_id: 4-char station ID
158         ztd: Zenith Total Delay (mm)
159         ztd_sigma: ZTD uncertainty (mm)
160         timestamp: Observation time
161         latitude: Station latitude (degrees)
162         longitude: Station longitude (degrees)
163         height: Ellipsoidal height (m)
164         met_obs: Optional meteorological observation
165
166     Returns:
167         IWVResult with all derived quantities
168     """
169     # Get meteorological data
170     if met_obs is None and self.met_db:
171         met_obs = self._find_nearest_met_obs(latitude, longitude,
172 timestamp)
173
174     if met_obs is None:
175         raise ValueError("No meteorological data available")
176
177     # Extrapolate pressure to GPS height
178     pressure = self._extrapolate_pressure(
179         met_obs.msl_pressure,
180         met_obs.height,
181         height,
182         met_obs.temperature,
183     )
184
185     # Calculate ZHD
186     zhd = self.calculate_zhd_saastamoinen(pressure, latitude, height
187 )
188
189     # Calculate ZWD
190     zwd = ztd - zhd
191
192     # Calculate mean temperature

```

```
191     tm = self.calculate_mean_temperature(met_obs.temperature,
192     latitude)
193
194     # Calculate IWV
195     iwv = self.calculate_iwv_from_zwd(zwd, tm)
196
197     # Propagate uncertainty
198     iwv_sigma = self._propagate_uncertainty(ztd_sigma, zwd, tm)
199
200     return IWVResult(
201         station_id=station_id,
202         timestamp=timestamp,
203         mjd=mjd_from_datetime(timestamp),
204         ztd=ztd,
205         ztd_sigma=ztd_sigma,
206         zhd=zhd,
207         zwd=zwd,
208         iwv=iwv,
209         iwv_sigma=iwv_sigma,
210         latitude=latitude,
211         longitude=longitude,
212         height=height,
213         pressure=pressure,
214         temperature=met_obs.temperature,
215         met_station_id=met_obs.station_id,
216     )
```

Listing 9.1: ZTD2IWV Core Implementation

Chapter 10

Station Management

10.1 Station Class

The station module (`stations/station.py`) provides unified station management supporting both XML and YAML formats.

```
1  """
2  Station management and XML/YAML parsing.
3
4  Replaces Perl STA.pm module.
5  Supports both XML (legacy) and YAML (preferred) station files.
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import dataclass, field
11 from pathlib import Path
12 from typing import Any
13 from xml.etree import ElementTree
14
15 import yaml
16
17 from pygnss_rt.utils.logging import get_logger
18
19
20 logger = get_logger(__name__)
21
22
23 @dataclass
24 class Station:
25     """GNSS station information."""
26
27     station_id: str
28     name: str | None = None
29     network: str | None = None
30     latitude: float | None = None
31     longitude: float | None = None
32     height: float | None = None
33     x: float | None = None      # ECEF X coordinate
34     y: float | None = None      # ECEF Y coordinate
35     z: float | None = None      # ECEF Z coordinate
36     receiver_type: str | None = None
37     antenna_type: str | None = None
38     use_nrt: bool = True
39     active: bool = True
40     metadata: dict[str, Any] = field(default_factory=dict)
41
42     def to_dict(self) -> dict[str, Any]:
43         """Convert to dictionary."""
```

```

44         return {
45             "station_id": self.station_id,
46             "name": self.name,
47             "network": self.network,
48             "latitude": self.latitude,
49             "longitude": self.longitude,
50             "height": self.height,
51             "x": self.x,
52             "y": self.y,
53             "z": self.z,
54             "receiver_type": self.receiver_type,
55             "antenna_type": self.antenna_type,
56             "use_nrt": self.use_nrt,
57             "active": self.active,
58         }
59
60     @classmethod
61     def from_yaml_dict(cls, data: dict[str, Any]) -> Station:
62         """Create station from YAML dictionary."""
63         return cls(
64             station_id=data.get("id", "").lower(),
65             name=data.get("name"),
66             network=data.get("network"),
67             latitude=data.get("latitude"),
68             longitude=data.get("longitude"),
69             height=data.get("height"),
70             x=data.get("x"),
71             y=data.get("y"),
72             z=data.get("z"),
73             receiver_type=data.get("receiver"),
74             antenna_type=data.get("antenna"),
75             use_nrt=data.get("use_nrt", True),
76             active=data.get("active", True),
77             metadata=data.get("metadata", {}),
78         )
79
80
81     class StationManager:
82         """Station management class."""
83
84         def __init__(self):
85             """Initialize station manager."""
86             self._stations: dict[str, Station] = {}
87
88         def load(self, filepath: Path) -> int:
89             """Load stations from file (auto-detect format).
90
91             Args:
92                 filepath: Path to XML or YAML file
93
94             Returns:
95                 Number of stations loaded
96             """
97             if filepath.suffix in (".yaml", ".yml"):
98                 return self.load_yaml(filepath)
99             elif filepath.suffix == ".xml":
100                 return self.load_xml(filepath)
101             else:

```

```

102         raise ValueError(f"Unknown file format: {filepath.suffix}")
103
104     def load_yaml(self, filepath: Path) -> int:
105         """Load stations from YAML file."""
106         with open(filepath) as f:
107             data = yaml.safe_load(f)
108
109             stations = data.get("stations", [])
110             for sta_data in stations:
111                 station = Station.from_yaml_dict(sta_data)
112                 self._stations[station.station_id] = station
113
114             logger.info(f"Loaded {len(stations)} stations from {filepath.name}")
115             return len(stations)
116
117     def get_station(self, station_id: str) -> Station | None:
118         """Get station by ID."""
119         return self._stations.get(station_id.lower())
120
121     def get_stations(
122         self,
123         network: str | None = None,
124         use_nrt: bool | None = None,
125         active: bool | None = None,
126     ) -> list[Station]:
127         """Get stations with optional filters."""
128         stations = list(self._stations.values())
129
130         if network:
131             stations = [s for s in stations if s.network == network]
132         if use_nrt is not None:
133             stations = [s for s in stations if s.use_nrt == use_nrt]
134         if active is not None:
135             stations = [s for s in stations if s.active == active]
136
137         return stations

```

Listing 10.1: Station Class Implementation

Chapter 11

Mathematical Foundations

11.1 PPP-AR Observation Model

The ionosphere-free linear combination for dual-frequency observations:

$$L_{IF} = \frac{f_1^2 L_1 - f_2^2 L_2}{f_1^2 - f_2^2} \quad (11.1)$$

$$P_{IF} = \frac{f_1^2 P_1 - f_2^2 P_2}{f_1^2 - f_2^2} \quad (11.2)$$

11.2 Kalman Filter State Vector

The PPP-AR state vector contains:

$$\mathbf{x} = \begin{bmatrix} \mathbf{r} \\ \delta t_r \\ \text{ZTD} \\ G_N, G_E \\ \mathbf{N} \end{bmatrix} \quad (11.3)$$

where:

- $\mathbf{r}$: Position coordinates (X, Y, Z)
- δt_r : Receiver clock offset
- ZTD: Zenith Tropospheric Delay
- G_N, G_E : Tropospheric gradients (North, East)
- $\mathbf{N}$: Carrier phase ambiguities

11.3 LAMBDA Ambiguity Resolution

The integer ambiguity search minimizes:

$$\|\hat{\mathbf{a}} - \mathbf{a}\|_{\mathbf{Q}_{\hat{\mathbf{a}}}}^2 = (\hat{\mathbf{a}} - \mathbf{a})^T \mathbf{Q}_{\hat{\mathbf{a}}}^{-1} (\hat{\mathbf{a}} - \mathbf{a}) \quad (11.4)$$

The ratio test validates the solution:

$$R = \frac{\|\hat{\mathbf{a}} - \mathbf{a}_2\|^2}{\|\hat{\mathbf{a}} - \mathbf{a}_1\|^2} > R_{threshold} \quad (11.5)$$

11.4 VMF3 Mapping Functions

The Vienna Mapping Functions VMF3 compute slant delays:

$$m_h(e) = \frac{1 + \frac{a_h}{1 + \frac{b_h}{c_h}}}{\sin e + \frac{a_h}{\sin e + \frac{b_h}{c_h}}} \quad (11.6)$$

$$m_w(e) = \frac{1 + \frac{a_w}{1 + \frac{b_w}{c_w}}}{\sin e + \frac{a_w}{\sin e + \frac{b_w}{c_w}}} \quad (11.7)$$

Chapter 12

Bernese Configuration

12.1 YAML Configuration Format

The BSW configuration uses YAML format for INP file customization:

```
1  # PyGNSS-RT BSW Configuration for PPP-AR (GE Mode)
2  # Network: IGS Core Stations
3  # Version: 1.4.0-GE
4
5  processing_mode: PPP_AR
6  gnss_systems:
7    GPS: true
8    Galileo: true
9    GLONASS: false  # Disabled due to BSW 5.4 CDSPP limit
10
11 bern_options:
12   PPP_GEN:
13     GPSEST:
14       # Multi-GNSS selection (GE = GPS + Galileo)
15       USE_G: '1'    # GPS enabled
16       USE_R: '0'    # GLONASS disabled
17       USE_E: '1'    # Galileo enabled
18       USE_C: '0'    # BeiDou disabled
19
20       # Troposphere modeling
21       TROPOS: DRY_VMF3
22       TROPOS_S: MENDES-PAVLIS
23
24       # Ambiguity resolution
25       AMBSTRAT: SIGMA
26       USEAR_G: '1'
27       USEAR_E: '1'
28       USEAR_R: '0'
29       GLORES: GNSS_BY_GNSS
30       LAMRES: GNSS_BY_GNSS
31
32       # OSB configuration
33       OSBREC_G: SYSTEM
34       OSBREC_E: SYSTEM
35
36   PPP_IAR:
37     GPSEST:
38       # Integer Ambiguity Resolution step
39       AMBSTRAT: SIGMA
40       USEAR_G: '1'
41       USEAR_E: '1'
42
43       # Ratio test threshold
44       MAXFRAC: '0.1'
```

```

45 SEARWID: '0.5'
46 RMSNARR: '0.03'

```

Listing 12.1: PPP-AR YAML Configuration Example

12.2 GE Configuration Rationale

Table 12.1: GE (GPS+Galileo) Configuration Benefits

Aspect	Setting	Rationale
Signal Type	CDMA only	Both GPS and Galileo use CDMA with clean integer properties
AR Method	GNSS_BY_GNSS	Independent AR per constellation then combine
OSB Mode	SYSTEM	System-wide biases (no FDMA IFBs)
Troposphere	VMF3	Vienna Mapping Functions for accurate ZTD

Appendix A

Complete Module Listing

Table A.1: Complete PyGNSS-RT Module List (75 files, 46,945 lines)

Package	Module	Lines
pygnss_rt/	cli.py	2,663
	__init__.py	42
	cron_hourly.py	156
core/	paths.py	289
	config.py	187
	exceptions.py	98
	orchestrator.py	445
processing/	daily_ppp.py	1,398
	networks.py	450
	orchestrator_main.py	2,891
	bsw_options.py	312
	nrddp_tro.py	567
	daily_crd.py	234
	neq_stacking.py	389
	station_merger.py	276
data_access/	campaign_archival.py	198
	product_downloader.py	1,212
	station_downloader.py	678
	ftp_client.py	456
	http_client.py	312
bsw/	ftp_config.py	189
	bpe_runner.py	1,089
	environment.py	234
	interface.py	178
	parsers.py	312
atmosphere/	rnx2snx.py	156
	ztd2iwv.py	917
stations/	inx2tec.py	234
	station.py	358
	station_info.py	289
	site_log_parser.py	445
	sta_file_writer.py	312
utils/	coordinates.py	178
	dates.py	331
	logging.py	156
	compression.py	189
	rinx.py	445
database/	multi_gnss.py	234
	models.py	312
	connection.py	178
	daily_data.py	267
	hourly_data.py	234

Appendix B

Installation and Usage

B.1 Installation from GitHub

```
1  # Clone repository
2  git clone https://github.com/georgisalene/pygnss_rt.git
3  cd pygnss_rt
4
5  # Create virtual environment
6  python3 -m venv .venv
7  source .venv/bin/activate
8
9  # Install package
10 pip install -e .
11
12 # Verify installation
13 pygnss_rt --version
```

Listing B.1: Installation Commands

B.2 Basic Usage Examples

```
1  # Process IGS network for a date range
2  pygnss_rt daily-ppp IG -s 2025-12-01 -e 2025-12-31
3
4  # Download products only
5  pygnss_rt download-products -d 2025-12-23
6
7  # Convert ZTD to IWV
8  pygnss_rt ztd2iwv /path/to/troposphere.TRP
9
10 # List stations in a network
11 pygnss_rt stations /path/to/IGS20.yaml --network IGS20
12
13 # Update BSW station file
14 pygnss_rt update-sta -n IG --output NEWNRT54.STA
```

Listing B.2: CLI Usage Examples

Appendix C

References

1. Bevis, M., Businger, S., Herring, T.A., Rocken, C., Anthes, R.A., Ware, R.H. (1992). GPS Meteorology: Remote Sensing of Atmospheric Water Vapor Using the Global Positioning System. *Journal of Geophysical Research*, 97(D14), 15787-15801.
2. Landskron, D., Böhm, J. (2018). VMF3/GPT3: Refined discrete and empirical troposphere mapping functions. *Journal of Geodesy*, 92, 349-360.
3. Teunissen, P.J.G. (1995). The least-squares ambiguity decorrelation adjustment: a method for fast GPS integer ambiguity estimation. *Journal of Geodesy*, 70, 65-82.
4. Dach, R., Lutz, S., Walser, P., Fridez, P. (2015). Bernese GNSS Software Version 5.2. *Astronomical Institute, University of Bern*.
5. Schaer, S. (2016). CODE's observable-specific signal bias (OSB) files: PPP-AR ready. *IGS Workshop 2016, Sydney*.